

▲↓ V2 Inverted Hash 512×512

> The sixth work in Marco Lazcano's pyramidal hash research series.

> ****Double-scale Sacred Pair inversion**** — 255 K2 active anchors per round —

> with ****superlinear SAC scaling**** and the tightest avalanche σ in the series.

>

> ****Author:**** Marco Lazcano — Decatur, GA, USA — roble99oak77@gmail.com — May 2026

⚠ Security Notice

This is a ****research prototype****. It has not been formally audited.

****Do not use in production systems or cryptographic protocols****

without independent peer review and formal cryptanalysis.

Overview

| Property | Value |

|-----|-----|

| Input block | 512 bytes |

| Internal state | 512 bytes (4096 bits) |

| Output | 512 bytes (4096 bits) |

| Permutation rounds | 24 |

| Chunk size | 512 bytes (full state, one block) |

| Anchor order | ****tip → base (INVERTED)**** |

| K1 region | dim 0–255 → unchanged (256 anchors) |

| K2 region | dim 256–510 → SP(x) active **** (255 anchors) **** |

| K1/K2 boundary | Dimension 257 |

| Sacred Pair | $SP(x) = (\sim x \ \& \ 0xFF)$ |

| Architecture | Sponge construction |

Key Finding — Superlinear SAC Scaling

When K2 active anchors double (127→255), SAC improves ****beyond proportional prediction****:

...

256×256 Original Hash: SAC = 0.98% over 2,048 bits

Proportional prediction: SAC = 1.96% over 4,096 bits (×2 expected)

V2 Inverted 512×512: SAC = 1.12% over 4,096 bits (actual)

Improvement over prediction: 1.96% - 1.12% = 0.84% better ★

...

Doubling the K2 region produces a **disproportionately stronger diffusion effect** because 255 complemented anchors (XOR distance = 0xFF each) create richer cross-anchor interactions across the larger 512-byte state space.

Statistical Test Results — Full 4096-bit Output

Test	Result	Threshold	Status
Basic functionality	All distinct	All distinct	✓
Determinism (200 trials)	0 errors	0 errors	✓
Avalanche mean	49.894%	50.00% ± 2%	✓ EXCELLENT
Std deviation ★	0.741%	< 5%	✓ BEST IN SERIES
Shannon entropy	7.9987 b/byte	> 7.98	✓ EXCELLENT
Entropy efficiency	99.98%	> 99%	✓ EXCELLENT
Chi-square ★	252.52	< 293.2 (95%)	✓ BEST IN SERIES
Pearson corr. avg (N=1000)	0.02528	< 0.05	✓ EXCELLENT
Pairs with corr > 0.10	186 / 130,816	(0.14%) ~0%	✓ EXCELLENT
S-box max differential	1.56%	< 6.25%	✓ SECURE
SAC deviation > 10%	1.12%	< 5%	✓ EXCELLENT
Collision resistance	0 / 400	0	✓ ROBUST

Research Program — Complete Series

#	Work	I/O bytes	K2 anchors	SAC	σ avalanche
1	Marco System	128×256	—	—	—
2	Marco's Theorem	—	—	—	—
3	Pyramidal Hash	256×512	64B	Never	— 2.26%
4	512×512 Inverted Hash	256B	Never	1.09%	1.104%
5	256×256 Original Hash	256B	127 ✓	0.98%	1.196%
6	V2 Inverted 512×512	512B	255 ✓✓	1.12%	0.741% ★

How It Works

1. Pyramidal Nonlinear Function — 512-byte base

```
```python
diff = SBOX[(seq[i+1] - seq[i]) mod 256] # AES S-box
diff = safe_byte(diff XOR anchor) # anchor mixing
diff = rotate_left(diff, (i mod 7) + 1) # position rotation
```
```

512 input bytes → 511 levels → 511 left anchors + 1 tip.

2. Inverted Output with Active K2

```
```python
K1/K2 processing at dimension 257
for i, anchor in enumerate(left_anchors):
 if i >= 256: processed.append(SP(anchor)) # K2 — 255 anchors
 else: processed.append(anchor) # K1 — 256 anchors

Inverted: tip first, anchors reversed
output = [tip] + processed[::-1]
```
```

3. Sacred Pair Properties

```
...
SP(x) = (~x) & 0xFF
Involution: SP(SP(x)) = x for all x
XOR distance: x XOR SP(x) = 0xFF (maximum possible)
...

```

4. Internal Permutation (24 rounds)

1. **Full 512-byte pyramidal mixing** — 255 K2 anchors complemented
2. **Linear diffusion** — each byte XOR-ed with the next
3. **Nonlinear substitution** — AES S-box on every byte

Installation

```
```bash
git clone https://github.com/your-username/v2-inverted-hash-512x512.git
cd v2-inverted-hash-512x512
python3 v2_inverted_hash_512x512_analysis.py
```
```

```
...
```

```
---
```

Usage

```
```python
from v2_inverted_hash_512x512 import V2InvertedHash
```

```
Full 4096-bit output (default)
h = V2InvertedHash()
h.update(b"Hello, world!")
print(h.hexdigest(512)) # 1024 hex chars = 4096 bits
```

```
256-bit output
h2 = V2InvertedHash()
h2.update(b"Hello, world!")
print(h2.hexdigest(32)) # 64 hex chars = 256 bits
```

```
Streaming
h3 = V2InvertedHash()
h3.update(b"first chunk")
h3.update(b"second chunk")
print(h3.hexdigest(512))
...
```

### ### Example outputs (first 32 hex chars)

```
...
```

```
b"" (empty) -> 0b7310e53f3534a741e63ccc8be96cab...
b"\x00" -> b3028382106b4c14eebfe1e4cfd81670...
b"\x01" -> 85f3ea796903d160a0df07fba3cff4b3...
b"\xff" -> 37fc056bb6ec75e44f972126e0f04029...
b"Hola Mundo" -> f15028ddd1d735a74fb6464abc6a5031...
b"Hello" -> 3f16564bed23d47a2928cc5b4b212f20...
b"hello" -> 33cfb21adc4ed857f0588fb7399cd56c...
b"hello " (space) -> d6663ffc26aa0969821dafb357ed42a0...
...
```

```

```

### ## Performance

> Pure Python 3 — not representative of a compiled implementation.

Input size	Time / hash
-----	-----
512 bytes	~5,951 ms
1 KB	~7,937 ms
2 KB	~11,893 ms
4 KB	~19,819 ms

A C/Rust implementation is estimated to be ~100× faster.

---

## ## Reproducing the Tests

```
```bash
python3 v2_inverted_hash_512x512_analysis.py
```

```
# Estimated runtimes:
# Pearson correlation (N=1000, 130,816 pairs): ~2 hours
# SAC (150 samples, 8 bits, 4096 output bits): ~2.5 hours
# Total: ~5-6 hours
```
```

---

## ## Known Limitations

- **No formal cryptanalysis** — differential, linear, algebraic attacks not studied.
- **Scaling law** — superlinear SAC property confirmed at 256→512 byte scale only.
- **Python only** — unsuitable for production performance.
- **No external audit** — cryptographic peer review pending.

---

## ## Academic Paper

 **V2 Inverted Hash 512×512: A Cryptographic Hash Function with Double-Scale Sacred Pair Inversion and Superlinear SAC Scaling**  
\*Marco Lazcano, May 2026\*  
\*(submitted for peer review)\*

---

## ## License

MIT License — © 2026 Marco Lazcano — see [LICENSE](LICENSE)

---

## ## References

1. Daemen & Rijmen (2002). \*The Design of Rijndael: AES\*. Springer.
2. Bertoni et al. (2011). \*Cryptographic sponge functions\*. IACR.
3. NIST (2015). \*SHA-3 Standard\*. FIPS PUB 202.
4. Lazcano, M. (2026). \*Marco System 128×256\*. Independent Research.
5. Lazcano, M. (2026). \*Marco's Theorem\*. Independent Research.
6. Lazcano, M. (2026). \*Pyramidal Hash 256×512\*. Independent Research.
7. Lazcano, M. (2026). \*512×512 Inverted Hash\*. Independent Research.
8. Lazcano, M. (2026). \*256×256 Original Hash\*. Independent Research.

---

<div align="center">

<sub>V2 Inverted Hash 512×512 — © 2026 Marco Lazcano — Research prototype · Not for  
production use</sub>

</div>